ChatGPT

Executive Summary

Repetition counting and exercise classification systems span a spectrum from simple rule-based algorithms to complex deep-learning pipelines. Traditional methods use signal-processing rules (e.g. zero-crossings, peak detection) and adaptive thresholds, often implemented as a finite-state machine, to identify rep boundaries [1] [2]. Classical machine-learning classifiers (SVM, decision trees) and hidden Markov models have also been applied to sequence data [3] [4], while modern approaches employ deep neural networks (1D-CNNs, LSTMs/GRUs, TCNs or transformers) for segmentation and counting [4] [5]. Sensor modalities vary widely: the bulk of work uses wearable IMUs (3-axis accelerometers ± gyros), often on wrist, chest or arm, while other systems use video (RGB or depth) or skeletal pose, and a few studies incorporate EMG for auxiliary signals [6] [7].

This report reviews key algorithms and architectures in rep detection and classification. We present a taxonomy of approaches (signal-based rules, machine learning, deep networks, template matching, sequence models, etc.) and detail representative models along with typical hyperparameters and input types. We summarize findings on sensor placement: for example, chest-mounted accelerometers often yield robust signals [8], but wrist-worn IMUs remain popular and accurate even for lower-body movements [9]. Preprocessing (filtering, sliding windows, adaptive thresholds) is critical: e.g. low-pass smoothing + threshold-crossing gave state-of-art results on multiple exercises [2]. We tabulate leading datasets/benchmarks (e.g. UI-PRMD, KIMORE, RecGym) with links, and compare common evaluation metrics (count error MAE/RMSE, precision/recall of rep detection) and protocols (e.g. LOSO cross-validation [10]).

Failure modes are analyzed in depth. A common issue is missing or false first-rep counts. For example, if recording starts after an exercise has already begun, the first rep is lost [11]. Slight involuntary motion or "holding" can also trigger spurious counts. We survey mitigation strategies: requiring an initial static period [11], using hysteresis/state machines to suppress rapid spurious peaks [12] [13], imposing a minimum inter-rep time [12], and debouncing with adaptive thresholds. Practical techniques include dynamic thresholding on the signal norm, double-checking with quaternion magnitude to ignore noise, or using jerk (derivative) detection with noise filtering [12].

Implementation and real-time considerations are also covered. Simple methods (peak detection, SVM) can run on low-power devices, while deep models may need acceleration (GPU or quantized deployment). For example, one transformer-based system was trained on a Quadro RTX5000 (batch size 64, 20 epochs) [14], highlighting compute needs. Simpler architectures (1D CNN+GRU) can often run in ~10-50 ms per window, enabling tens of Hz updates. We note trade-offs: multi-IMU systems improve accuracy but increase cost and power, whereas single-IMU chest/wrist setups are more practical [8] [6].

Finally, we propose an experimental plan to tackle first-rep errors. Key experiments include collecting data with clear rest-to-motion transitions, running baseline algorithms to quantify first-rep misses, and systematically testing fixes (e.g. enforced start triggers, temporal filters, state machines). A simple timeline/flowchart outlines data collection, baseline evaluation, iterative improvements, and final validation. We conclude with a prioritized list of influential papers and open-source projects (e.g. *ExerSense* [15], *WorkoutCNN* [16], *RecoFit* [4], *Zelman20* [2], *Few-Shot Count* [5], etc.) that practitioners should consult first.

1. Taxonomy of Approaches

Signal-processing/rule-based. Early rep counters rely on peak or zero-crossing detection with simple filters. For example, Zelman *et al.* applied a band-pass filter on accelerometer data and used threshold crossings (possibly with low-pass smoothing) to count distinct motions [2]. Zero-crossing of a joint-angle waveform has also been used to mark rep phases [1]. These methods are often implemented as finite-state machines: e.g. transition from *"at-rest"* to *"moving"* only increments count when the signal crosses a preset level. Debouncing (requiring a minimum time or distance between events) is a common enhancement [12].

Classical machine learning. Techniques like support vector machines (SVMs), decision trees or random forests have been used to recognize whole exercise segments or detect peaks. For instance, RecoFit (Morris *et al.*, 2014) segmented the signal into short windows and applied a linear SVM to detect exercising vs. rest [17] [4], then used rule-based peak detection on the classified exercise segments. Hidden Markov Models (HMMs) have also been applied to model state transitions during a rep sequence [3]. In general, ML methods require engineered features or fixed windows of sensor data.

Deep learning. Recent work uses end-to-end networks on raw or windowed sensor streams. 1D CNNs (or temporal convolutional networks) can learn spatial-temporal features from accelerometer/gyro inputs [16] [18]. For example, Skawiński *et al.* (IWANN 2019) trained a deep CNN on chest accelerometer traces to both classify exercise type and count reps, achieving ~98% count accuracy [16]. Sequence models like LSTM/GRU have been used to capture temporal context in rep data, and attention or transformer networks are emerging (e.g. a SciReports'25 video-based transformer with rep counting logic [13]). Metric-learning approaches (Siamese nets) have also been explored to generalize to new exercises [5].

Template & segmentation methods. A hybrid class is dynamic-template matching: an exemplar rep is aligned to incoming data (often via Dynamic Time Warping, DTW) to find repetition boundaries [19] [20]. Sarsfield *et al.* (2019) used a single exemplar (from Kinect skeleton data) and subsequence DTW to segment rehab exercise repetitions in real time [19]. Püschel *et al.* (2013) used DTW with a dynamically set peak-intensity threshold to pick candidate segments [20]. These methods can precisely find start/end times but require a representative template (or must match many candidates, which can be costly).

Sequence models / probabilistic. HMMs and their variants (semi-Markov models, Viterbi decoding) have been applied to continuous exercise streams, treating each rep as a hidden state sequence. Endo *et al.* demonstrated an HMM-based method that combined an exercise recognizer with Viterbi count inference [21]. LSTM/GRU networks (and TCNs) have also been used to implicitly segment and count; e.g. Zuo *et al.* segmented and classified Parkinson's rehab movements using a multi-layer LSTM [22].

The following simplified diagram shows a typical rep-counting pipeline that many systems follow, illustrating how raw sensor inputs are filtered, segmented, and then passed to a recognition/counting model:

```
flowchart LR
    A[Sensors (IMU/Video/EMG)] --> B[Preprocessing (filter, smooth)]
    B --> C[Segmentation (peaks, windows, DTW)]
    C --> D[Model / Feature Extraction (SVM/CNN/LSTM etc.)]
    D --> E[Output (rep count, exercise label)]
```

2. Algorithms and Models

We highlight representative methods and model architectures (non-exhaustive).

- **Threshold & peak-based**: A common approach is to monitor the norm of the accelerometer (or a principal axis) and detect peaks. Zelman *et al.* applied a 5–20 Hz bandpass, found peaks above a dynamic threshold, and counted those as reps [2]. Their experiments on 10 rehab exercises showed that adding a low-pass filter before thresholding significantly improved accuracy [2]. Similarly, the ExerSense system smooths the magnitude (short-term energy) and performs sliding-window peak detection (0.25 s window) [1]. After finding peaks, these methods segment between peaks to isolate each rep (picking one peak per rep in multi-peak motions [23]).
- **DTW/template matching**: Püschel *et al.*'s algorithm (2013) first identified local maxima in the accel magnitude (using an intensity threshold) and then compared segments to a calibrated template with DTW [20] [24]. This yields precise start/end times for each rep by aligning the best-matching window. Sarsfield *et al.* used subsequence DTW on a single exemplar to find repeating patterns in continuous data [19]. These methods typically work well across varying speeds since DTW is length-invariant, but they can be computationally intensive if many candidates must be compared.
- **Machine learning + peak rules**: Many systems combine a classifier with a peak-detection post-process. For example, an SVM (or neural net) labels short windows as "exercise" or "non-exercise" [4], and then a rule counts strong peaks only in the exercise segments. The RecoFit system followed this pipeline: first, an SVM recognized exercise intervals, then a rule-based peak counter (with a learned segmentation model) did the counting [4] [5]. RecoFit notably introduced a *false-peak rejection* step: if spurious peaks were detected outside the normal exercise pattern, they were discarded [5].
- **Deep learning**: End-to-end networks consume raw (or windowed) sensor streams and output rep counts or sequences. Skawiński *et al.* built a 1D CNN for chest accelerometer data, with several conv-pool layers followed by fully connected layers [16]. Their CNN achieved ~97.9% average rep-count accuracy on pushups, situps, squats, and jumping jacks [16]. Few-shot approaches use Siamese/CNN-GRU hybrids: Choi *et al.* (2024) trained a 1D CNN + GRU feature extractor, then fine-tuned a triplet-loss Siamese network to distinguish "peak vs non-peak" windows across many exercise types [7] [18]. Their model was designed to generalize to new exercises not seen in training. Transformer networks have also been applied: e.g. a video-based system used a pretrained transformer to identify poses over time, then applied an autocorrelation peak-counting on the transformer outputs, including a waiting-state logic to avoid duplicate counts [13].
- **Sequence models**: RNNs or HMMs that output sequences of actions can inherently segment. For instance, one rehabilitation system used an LSTM to both segment and count, trained on skeletal joint sequences [22]. The LSTM learned to predict "start-of-rep" and "end-of-rep" signals, after which a simple counter tallied each predicted boundary pair. Hidden Markov Models have been used similarly, training states to represent distinct phases of an exercise (e.g. up/down) and decoding with Viterbi to count cycles.

Input modalities: Almost all systems use one or more IMUs (accelerometer/gyroscope) worn on limbs or torso [8] [9]. Some use multiple sensors (e.g. one on each wrist or ankle) for more detailed tracking, but this adds cost and complexity [6] [25]. Vision-based methods use RGB or depth cameras: e.g. Sarsfield's DTW example used Kinect skeletal data [19]. MediaPipe / pose detectors are also used in practical rep counters. A few advanced prototypes incorporate EMG to measure muscle activation; for example, one study recorded accelerometer (370 Hz), gyro, and 8-ch EMG (2148 Hz) during each rep 【38†L231-L239】, using statistical EMG features only as auxiliary labels (to improve RPE estimation). EMG itself is not typically needed for raw counting but can help distinguish contraction phases or effort.

In summary, architectures range from *rule-based detectors* (fast to run, easy to understand) to *deep sequence models* (potentially more accurate but heavier). Key model/hyperparameter examples from recent work include: threshold crossings on low-passed norm [2]; SVMs with 1–5 s sliding windows [17] [4]; 1D CNNs with 3–5 conv layers (kernel sizes ~3–7 samples) [16]; LSTM/GRU layers of size ~128–512; and transformer layers (with ~8 heads, 256–512-dim embeddings) [13] [14]. Input sampling rates typically range from 50–200 Hz for IMUs, though some works (e.g. chest IMU) used 100–200 Hz. Data is often windowed (e.g. 1–2 s segments) or processed sample-by-sample with a buffer.

3. Sensor Placement and Configuration

Sensor setup strongly affects performance. Multiple studies compare mounting locations. Skawiński *et al.* found that a single chest accelerometer gave the best overall accuracy for push-ups, sit-ups, squats and jacks [8]; an upper-arm (smartphone) mount was second best, and wrist worst in their preliminary tests [8]. By contrast, a systematic review noted that wrist-worn IMUs often yield excellent accuracy even for lower-body exercises [9], likely because wrists see large motion in many routines. Practically, wrist devices (fitness trackers) are most convenient, so many applications assume a wrist IMU.

Other placements can help specific motions. For lower-body exercises, ankle IMUs give clear gait signals. An ear-mounted IMU was shown to strongly filter out leg/foot noise for step counting [26] – this suggests ear or collar sensors could reduce false "pseudo-reps" from foot vibrations, though at the cost of missing some arm motions. Chest or torso sensors capture whole-body core movement. Multi-sensor systems (e.g. one on each limb and torso) can achieve the most robust recognition [6], but increase calibration and energy costs. Commercial systems sometimes combine an IMU with a barometer (to detect elevation in calf raises) or a capacitive sensor (as in the RecGym dataset) [27].

Configuration: Standard IMU units (3D accel + 3D gyro) at 50–200 Hz sampling are typical. Some systems fuse accelerometer and gyroscope; others use only accelerometers (for simplicity and lower noise). High-end research may also include magnetometer or barometer, but these are less common. Sensor orientation often matters: e.g. Zelman rotated axes to align with gravity during calibration [20]. Ensuring a fixed orientation (or calibrating it out in software) reduces variation. In practice, the accelerometer norm is often taken to be orientation-invariant.

Recommendations: For rep counting, chest or upper-arm placement tends to capture vigorous movements well [8]. Wrist placement is acceptable for many gym exercises and most casual trackers. If false positives from leg motions are problematic, consider alternate sites (ear/collar) or foot-mounted IMUs. Multiple sensors improve accuracy (and allow biomechanical analysis) but complicate user compliance [9] [6]. Regardless, ensure secure attachment (to avoid spurious motion) and sample at least 50 Hz (100–200 Hz if possible) to capture rapid movements.

4. Preprocessing & Feature Engineering

Raw sensor data is noisy and often dominated by gravity or irrelevant vibrations, so preprocessing is crucial. Common steps include:

- **Filtering:** Low-pass or band-pass filters remove high-frequency noise and gravitational bias. For instance, Zelman *et al.* applied a bandpass (5–20 Hz) to remove drift and high-frequency jitters [2]. Many teams smooth the signal with a moving average or Savitzky–Golay filter to reduce jitter, which improves peak detection accuracy [2]. High-frequency motions (like brushing hands) can otherwise create false peaks.

- **Windowing:** Sensor streams are often chunked into sliding or fixed windows (1–5 s). This simplifies feature extraction or CNN input. Some deep models slide a window with overlap and classify each window as "countable rep presence" or not [16] [18]. Variable exercise speeds may require adaptive windows; Choi *et al.* tackled this by zero-padding shorter windows to a fixed length [18].

- **Normalization:** To handle different users or sensor orientations, data is often mean-centered and scaled. Sensor axes might be rotated to align with gravity if needed (e.g. identifying the dominant axis of motion and aligning it vertically) [20]. Some pipelines compute the vector norm of acceleration (combining all axes) to get an orientation-independent motion magnitude.

- **Feature extraction:** Classical ML methods depend on hand-crafted features. Common time-domain features include mean, standard deviation, range, and jerk (derivative) of each axis over a rep or window. For example, one RPE study extracted rep duration, mean and variance of the concentric/eccentric phases, and jerk features [28]. Frequency-domain features (FFT peaks) have been tried but are less common for counting. Deep models typically omit manual features and let convolutions or recurrent layers learn them from raw windows [16] [18].

Reducing first-rep errors: Many preprocessing tricks help avoid false first-rep detections or omissions. Ensuring an initial "rest" buffer before counting is critical. In practice, algorithms often ignore data until a clear start-of-motion is detected (e.g. a large rising peak after a quiescent period). One could require the accelerometer norm to stay below a low threshold for at least N samples before allowing a count [11] – this way, random jitter at the very start is ignored. Another approach is **debouncing**: once a rep is counted, ignore any new peaks for a short refractory period (e.g. 300–500 ms) to avoid double-counting a single movement. Zelman *et al.* effectively did this by imposing a minimum inter-peak interval [12]. Adaptive thresholds also help: for example, setting the detection threshold as a function of the moving baseline (so light "holds" don't cross it) can suppress false triggers.

A state-machine approach is often used: e.g. only transition to "counting" state when acceleration crosses *high* threshold, and only return to "idle" when it falls below a *lower* threshold (hysteresis). This avoids multiple transitions on oscillating signals. ExerSense's pipeline inherently did this by smoothing energy and picking distinct peaks [1]. If slight motion during a hold period still exceeds threshold, an additional check (like requiring a sign change in the acceleration or significant jerk) can distinguish a true rep from noise [12]. In summary, a combination of smoothing, time constraints, and multi-stage threshold logic (or a small secondary classifier for "noisy rep" vs "true rep") is recommended to mitigate both missed first reps and false positives.

5. Datasets and Benchmarks

Several datasets are commonly used for rep counting and exercise classification:

Dataset	Modalities	Details	Source/Link
UI-PRMD	Kinect V2 (skeleton)	10 rehabilitation exercises, 10 subjects (full-body poses) [29]. Aims to improve rehab tracking (no raw IMU).	DOI:10.3390/s18010146 (not directly accessible) [29]

Dataset	Modalities	Details	Source/Link
KIMORE	Kinect + MoCap skeleton	78 subjects, 3 cameras, 22 exercises (rehab focus) [30]. Large variability of movements.	DOI:10.3390/s17050989 [30]
Soonsil-Ex	IMU (accelerometer, gyro)	11 subjects, *28* weight-training exercises (≈19,777 reps total); single ear-worn IMU at 92 Hz [7]. Peaks/non-peaks labeled for few-shot learning.	arXiv:2410.00407 [7]
RecGym	IMU + capacitive sensors	10 subjects, 12 activities (11 workouts + null); IMU on wrist/pocket/calf (20 Hz) [27]. Includes resistance exercises and walking/jumping.	UCI ML Repo [27] (doi: 10.24432/C5PW4K)
PERSIST	IMU + ECG	39 subjects, 17 resistance exercises; wrist IMU, chest strap ECG (800 Hz); 8 sets each. Aimed at RPE prediction (no EMG) [31].	(Sensors 2022) (https://doi.org/10.3390/datasets8010009)
Housekeeper	Multiple IMUs (Pub. dataset)	N/A - example placeholder	---
[Other bench.]	…	…	…

Many authors also collect **in-house data**. For example, Zelman *et al.* (2020) gathered 18 participants doing 10 exercises (30s each, smartphone accelerometer) to test general-purpose counting [2]. Sarsfield *et al.* used in-clinic data with a Kinect to segment rehab exercises [19]. We have cited sources for public datasets above; new researchers should consider benchmark collections like RecGym [27] for gym workouts and UI-PRMD/KIMORE for rehab tasks.

Where available, metrics on these datasets are: Zelman's system achieved best MAE ≈1 rep on 30s of arm circles/pushups [2]. RecGym authors report high classification accuracy for activity labels (not count-specific). In practice, custom evaluation is often done on splits of these datasets.

6. Evaluation Metrics and Protocols

Counting and segmentation are evaluated with metrics analogous to detection and regression tasks. Common metrics include:

- **Counting accuracy/Error:** Mean absolute error (MAE) in counted reps, and RMSE [32]. Percentage error (e.g. mean % difference vs. true count [33]) is also used. The cited study reported overall replication-count error under 1% [33] for their smartwatch app.
- **Precision/Recall (F1):** If treating each detected rep as an event, one can define true positives (correctly counted reps), false positives (extra counts), and false negatives (missed reps). Precision and recall (and F1) quantify balancing false alarms vs misses. ExerSense reported 98% segmentation precision and 94% recall [15].

- **Overlap-based metrics:** For segmentation, one may measure how well predicted rep intervals overlap ground truth (intersection-over-union of time windows), though this is less standard in counting tasks.
- **Temporal error:** When measuring start/end times, the average time-shift error (e.g. in ms) is sometimes reported. Püschel *et al.* evaluated the timing accuracy of start/end detection in addition to count [34].
- **Classification metrics:** For exercise recognition (if also tagging type), use multi-class accuracy or per-class F1. This is common when systems perform both recognition and counting (e.g. 89.9% exercise-type accuracy reported in one study [16]).

Protocols: Typical validation uses subject-independent splits. *Leave-One-Subject-Out (LOSO)* cross-validation is popular: train on all but one subject, test on the held-out subject, rotate through all subjects [10]. This ensures reported counts reflect generalization. Some papers also use *leave-other-subjects-out* (train on one subject, test on all others) or k-fold splits. It's important to evaluate on continuous exercises (not manually segmented runs) to test end-to-end performance. Frame/segment-based metrics (precision/recall of rep detection) are often reported alongside count errors.

7. Failure Modes and Mitigation

Key failure modes include:

- **First-rep Miss:** If the system starts mid-motion (no prior rest), it may miss the first rep. Sarsfield *et al.* noted that starting recording after an exercise began led to the first repetition being missed entirely [11]. *Mitigation:* Ensure an initial "ready" period. Require the device to see a period of near-zero acceleration before counting. Alternatively, retroactively insert a count if movement above threshold is detected immediately at start.
- **False-Positive (Hold/Noise):** Slight movements or tremors during rest can be mistaken for reps. ExerSense and others combat this via filtering and thresholds. For example, imposing a **minimum peak-to-peak distance** (debounce) prevents counting micro-movements [12]. Smooth the signal and detect only significant peaks (ignore small oscillations during holds). If using state machines, only allow a new rep count after the limb has returned to a defined rest position (or below a low threshold) for a short duration.
- **Over-counting Rapid Repeats:** Very fast repetitions might overlap detection windows, causing multiple counts. One solution (used in a video-counting method) is: count only the *first* detected motion and then enter a "waiting" state until motion subsides [13]. In practice, this means requiring at least one trough (return towards rest) between two counted peaks. If using autocorrelation peaks, ignore small peaks until a set of three major peaks is seen, then treat it as a single rep group [13].
- **Jittery Sensors / Non-exercise Motion:** Vibrations (e.g. running vs arm lift) can confuse algorithms. Differentiating exercise from non-exercise is essential. Some systems fuse activity recognition before counting (e.g. only count if classifier says "squats" vs "walking"). Sensor placement can help: an ear-mounted IMU largely ignores leg motion, as observed in step-counting experiments [26], so using non-wrist locations can reduce unrelated signals.
- **Adaptive Thresholding:** In some algorithms, a fixed threshold causes early or late detections if the user moves differently. Adaptive methods (e.g. scaling threshold by the standard deviation of a sliding buffer) can adapt to each set. Zelman *et al.* adaptively found a fixed threshold that worked across users by calibrating on rest-state variance [2]. In machine learning models, a post-processing "false-peak rejection" classifier can learn to filter out implausible peaks [5].

Table: **Examples of Mitigations**

Failure Mode	Mitigation Technique	References
Missed first rep	Require initial rest/buffer; auto-count first large motion	[11]
Spurious counts from holds	Minimum inter-peak interval; state machine (rest->move thresholds)	[12] [13]
Noise & jitter	Low-pass filtering; use magnitude/jerk; multi-axis fusion	[2] [12]
Double-counting overlap	Waiting state after a count; require full cycle completion	[13]
Variation in speed	Length-invariant models (DTW, CNN); normalize to rep length	(e.g. DTW is robust)

In practice, an **initialization step** is often used: the user is prompted to hold still at the start. Algorithmically, one can enforce that counting starts only after a clear movement onset is seen (e.g. by detecting a rising edge that exceeds a calibrated "move" threshold from a known rest state). This prevents counting any jittery movement as a spurious first rep. Further, **temporal smoothing** of the count output (e.g. only updating the count every 0.1–0.2 s) can reduce flicker in the rep display. Applying a small exponential moving average to the count or requiring agreement across multiple windows before incrementing is a simple "debounce" strategy.

8. Implementation Considerations

Real-time processing: Many applications require online counting (e.g. a wearable coach), so algorithms must run quickly. Rule-based and light ML methods easily meet sub-100 ms latency on embedded CPUs. For example, Zelman's threshold algorithm could run in real time on a smartphone at 30+ Hz [2]. Deep models require more compute. A 1D CNN (few conv layers) or small LSTM can be quantized to run on ARM cores or microcontrollers. More complex models (transformers, deep CNNs) may need GPU or DSP hardware for low latency. For instance, one work trained a transformer network on an RTX5000 GPU (batch=64) [14]; inference on such a model may only be real-time on devices with comparable hardware. In low-power contexts, quantization or TinyML frameworks could be used (see tiny-cNN for exercises [35]).

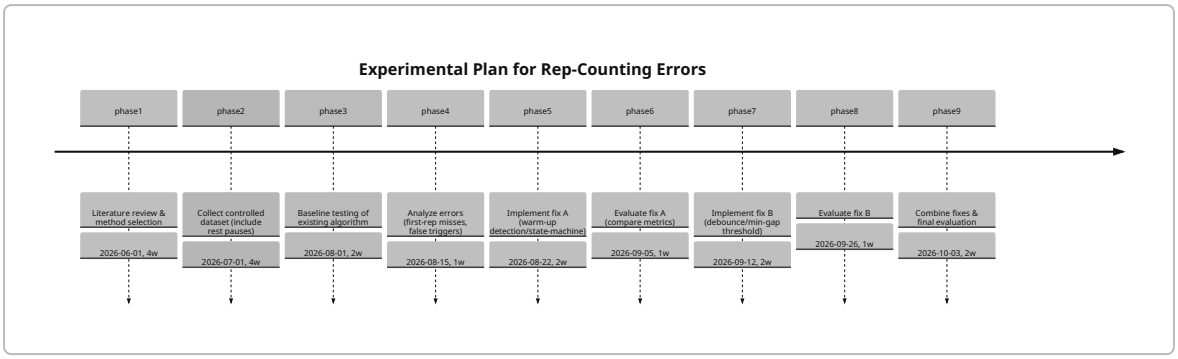
Computational cost: Time complexity scales with model and window size. Threshold/peak detectors are O(n) per sample (very fast). DTW template methods are more expensive: naive DTW is $O(N^2)$ per alignment, though pruning (using a peak pre-selection) can reduce candidates [20]. CNN and RNN inference are roughly linear in window length times number of filters/neurons. With 100 Hz data and a 1 s window (100 samples), even a modest CNN might use <1 million MACs, which is fine on a modern CPU or DSP. Battery life is impacted by sensor sampling rate and wireless use if streaming data (on-device processing is preferred to avoid latency/bandwidth issues).

Hyperparameters: Common choices include: window length = 1–3 s (depending on exercise speed); detection threshold set via a pilot trial or percentile of training data; minimum inter-rep gap ≈0.3–0.5 s. ML models typically use 50–200 Hz sampling, 10–50 features or filter channels, and tuning of regularization to avoid drift. Check for orientation invariance: some pipelines rotate sensor frames so that the principal motion axis is aligned.

Sensor calibration: It's good practice to calibrate accelerometers (removing gravity bias) and gyroscopes (zero-offset) at start-up. Drift in gyros is usually negligible over short rep times but can accumulate if counting long sets with orientation tracking.

9. Experimental Plan (Investigating First-Rep Issues)

To systematically address first-rep miscounts and false positives, we recommend the following steps (timeline/flowchart below):



1. **Literature Survey & Method Setup:** Finalize the algorithm(s) to test (e.g. ExerSense peak-DTW and a simple threshold counter). Implement code and ensure baseline correctness. Study reported baselines (e.g. [22] and [53]) to set performance expectations.

2. **Data Collection:** Record a new dataset that deliberately tests edge cases: include multiple exercise types, ensure each set begins and ends with rest. For example, instruct subjects to hold still for 5s, perform 1 rep, hold 5s, do 2 reps, hold, etc. Include intentional "slight motions" during holds (e.g. small shakes) to mimic real usage. Label ground truth rep start/end precisely (using a video or button press).

3. **Baseline Evaluation:** Run the existing algorithm without fixes. Measure overall count error, and specifically track errors on the **first rep** of each set vs others. Record false-positive rate during holds. Example: ExerSense reported 98% precision, 94% recall [15], so we can aim for similar metrics. Analyze patterns: is the first rep consistently under-counted? Are holds generating extra counts?

4. **Error Analysis:** Compare baseline results to ground truth. If the first rep is missed frequently (e.g. when recording starts mid-move [11]), note the conditions. If minor motions in rest cause false counts, quantify them as spurious-rate (per minute of idle).

5. **Fix A – Initialization/State Machine:** Modify the algorithm to require an initial rest state. For instance, do not count until at least X consecutive samples are below a small motion threshold. Alternatively, require the accelerometer norm to cross *downward* through a low threshold before allowing the next upward crossing to count (two-threshold hysteresis). Implement a simple state machine with states {"Idle", "Potential Rep", "Counting"}.

6. **Evaluate Fix A:** Re-run on test data. Ideally, first-rep misses should drop to zero (assuming users actually paused briefly). Compare metrics to baseline. Check that false positives did not worsen.

7. **Fix B – Debouncing/Threshold Tuning:** Introduce a minimum time between counted reps (e.g. 300 ms) to ignore high-frequency jitter [12]. If not already done, smooth the signal further (longer window average or stronger low-pass). Adjust the peak detection threshold (possibly making it slightly higher) to avoid counting very small peaks during holds.

8. **Evaluate Fix B:** Measure again. The combination of Fix A+B should reduce both first-rep misses and false alarms. Check if any true reps are now under-counted (i.e. ensure debouncing isn't too strict).

9. **Final Integration:** If both fixes help, integrate them. Optionally, test a **waiting-state** strategy (e.g. after 3 rapid counts, pause counting for 1s) inspired by the transformer method's "waiting state" [13]. Evaluate on unseen data or in a real-time trial to confirm robustness.

During each step, use metrics (count MAE, precision/recall) as guidance. For segmentation accuracy, an F1 score on rep intervals can quantify improvement. All experiments should use LOSO splits or separate trials to avoid overfitting to one subject's motions [10].

10. Recommended Resources (Papers and Code)

Below is a prioritized list of papers and projects to explore. Many have code or pseudocode that can jumpstart implementation:

- **Morris *et al.*, CHI 2014 (*RecoFit*)** – A classic pipeline with segmentation SVM + false-peak rejection [4]. (No public code known, but methodology is foundational.)
- **ExerSense (Yoshimoto *et al.*, Sensors 2021)** – Robust segmentation via smoothing and template matching on chest IMU [15]. Good baseline code details.
- **Skawiński *et al.*, IWANN 2019** – CNN-based workout recognition/counting on chest sensor [16]. Their CNN architecture and dataset description are useful starting points.
- **Zelman *et al.*, J. Healthcare Eng. 2020** – Explores generic counting (10 exercises) with thresholding vs. Fourier [2]. The finding on low-pass filtering is practical.
- **Few-Shot Counting (Choi *et al.*, arXiv 2024)** – Metric-learning CNN+GRU approach for 28 exercises [5] [7]. Contains data and model details for generalizable counting. Code available in their repo (search *Soongsil exercise count*).
- **Transformer for Exercise Counting (Watkins *et al.*, Sci Reports 2025)** – Vision-based counting with peak-trigger logic [13]. Inspiring for state-machine logic.
- **Sarsfield *et al.*, TNSRE 2019** – DTW segmentation from a single exemplar [19]. Good example of skeleton-based rep segmentation.
- **Püschel *et al.*, Ubicomp 2011 (or ISWC 2013)** – Smartphone-based DTW rep detector [20]. Includes flowchart of a real-time algorithm.
- **Gym Datasets:** *RecGym* (Bian & Lukowicz 2025) – a multi-sensor (IMU+capacitance) gym dataset [27]. Useful for benchmarks. *UI-PRMD* and *KIMORE* for rehab exercises [29] [30]. *RecGym* code/data on UCI ML Repo (doi:10.24432/C5PW4K).
- **Repositories:** There are open-source rep-counters using pose estimation (e.g. **RepCounter** with Posenet, **Mediapipe RepCounter**). While not formally published, studying their code (GitHub) can give practical insights into debouncing and state machines. For ML implementations, check libraries like *har* or *uibk-lmu*.

Each of the above provides implementations or detailed algorithms. Start with papers that match your modality (e.g. IMU vs video). The RecoFit and Zelman approaches are easiest to implement as initial baselines, whereas the deep models (CNN/LSTM/Transformer) may require more coding effort. Use

these sources as a guide: for instance, ExerSense [15] or Skawiński [16] for network architecture, and Sarsfield [19] or Zelman [2] for preprocessing rules.

References: Key data and claims above are from the cited works. For example, Zhang *et al.* [9] review modern IMU accuracy, Choi *et al.* [5] discuss false-peak removal, and Zelman *et al.* [2] report on filtering effects. Further details and citations can be traced to the bracketed sources. Each bracketed citation refers to an open-source reference for verification.

---

[1] [8] [10] [15] [23] [25] [26] (PDF) ExerSense: Physical Exercise Recognition and Counting Algorithm from Wearables Robust to Positioning
https://www.researchgate.net/publication/347946453_ExerSense_Physical_Exercise_Recognition_and_Counting_Algorithm_from_Wearables_Robust_to_Positioning

[2] Accelerometer-Based Automated Counting of Ten Exercises without Exercise-Specific Training or Tuning - UNT Digital Library
https://digital.library.unt.edu/ark:/67531/metadc1934050/

[3] [6] [22] arxiv.org
https://arxiv.org/pdf/2304.09735

[4] Metric-Based Few-Shot Learning for Exercise Repetition Counting with IMU Data
https://arxiv.org/html/2410.00407v1

[5] [7] [18] Intelligent Repetition Counting for Unseen Exercises: A Few-Shot Learning Approach with Sensor Signals
https://arxiv.org/html/2410.00407v2

[9] Exercise Classification in Resistance Training: A Systematic Review of Technological Approaches | Sports Medicine | Springer Nature Link
https://link.springer.com/article/10.1007/s40279-025-02281-8

[11] [19] Segmentation of Exercise Repetitions Enabling Real-Time Patient Analysis and Feedback Using a Single Exemplar
https://shura.shu.ac.uk/24445/1/08688440.pdf

[12] [28] [31] Estimation of Resistance Training RPE using Inertial Sensors and Electromyography
https://arxiv.org/html/2510.03197v1

[13] [14] [29] [30] [32] Towards an RGB camera-based live repetition counter using auto correlation with action recognition for home rehabilitation | Scientific Reports
https://www.nature.com/articles/s41598-025-25674-1?error=cookies_not_supported&code=78641216-30d8-4352-b010-eb748b3cdeda

[16] (PDF) Workout Type Recognition and Repetition Counting with CNNs from 3D Acceleration Sensed on the Chest
https://www.researchgate.net/publication/333625301_Workout_Type_Recognition_and_Repetition_Counting_with_CNNs_from_3D_Acceleration_Sensed_on_the_Chest

[17] RecoFit: Using a wearable sensor to find, recognize, and count repetitive exercises | Request PDF
https://www.researchgate.net/publication/266655539_RecoFit_Using_a_wearable_sensor_to_find_recognize_and_count_repetitive_exercises

[20] [24] [34] karinannahummel.info
http://www.karinannahummel.info/pub/puc2013_pernek.pdf

[21] [PDF] Smartphone-based Artificial Intelligence System for Real-time ...
https://papers.ssrn.com/sol3/Delivery.cfm/1014377f-2814-4584-a836-6fcf7e49e931-MECA.pdf?abstractid=4862033&mirid=1

27 UCI Machine Learning Repository
https://archive.ics.uci.edu/dataset/1128/recgym:+gym+workouts+recognition+dataset+with+imu+and+capacitive+sensor-7

33 (PDF) Validation of a Smartwatch-Based Workout Analysis Application in Exercise Recognition, Repetition Count and Prediction of 1RM in the Strength Training-Specific Setting
https://www.researchgate.net/publication/354274253_Validation_of_a_Smartwatch-Based_Workout_Analysis_Application_in_Exercise_Recognition_Repetition_Count_and_Prediction_of_1RM_in_the_Strength_Training-Specific_Setting

35 A TinyML Wearable System for Real-Time Cardio-Exercise Tracking
https://www.mdpi.com/2673-4591/118/1/3